Lab 10 – Worksheet

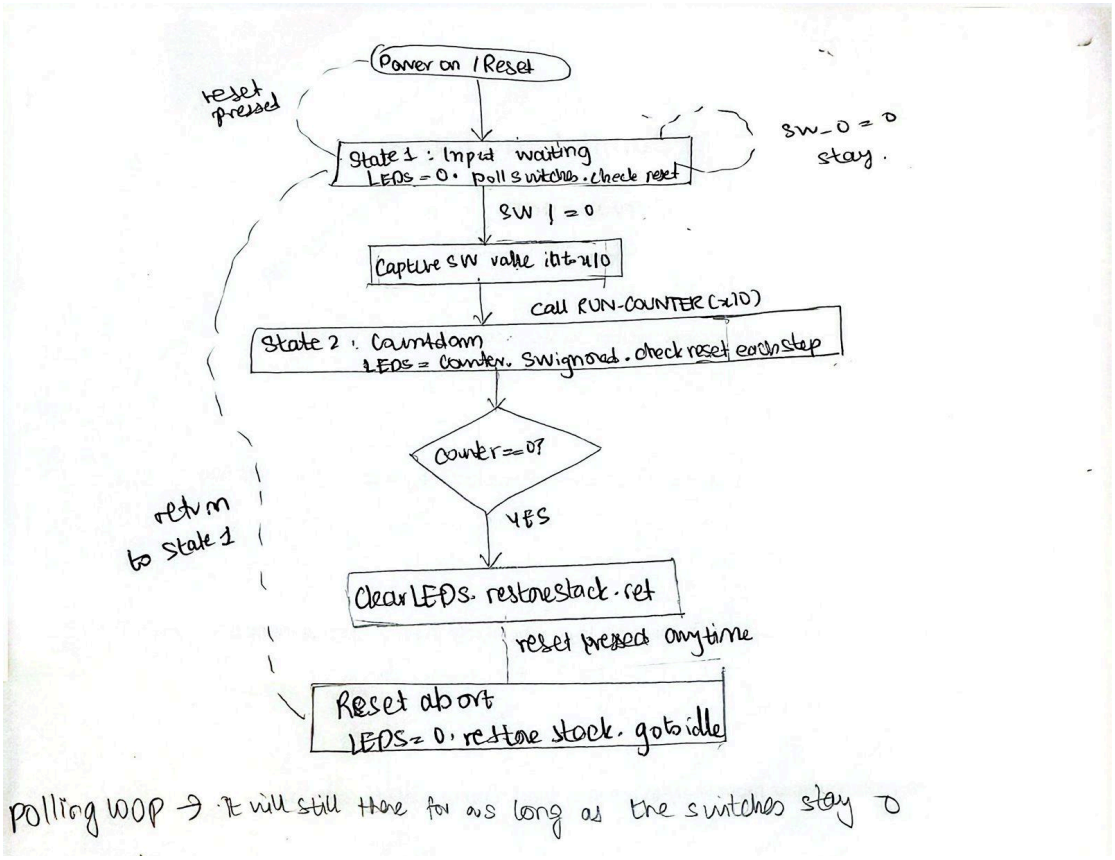
Name: Myrah Hussain Romaisa Shazad	ID: mh10074 rb10107	Section:

Note: Assumptions and logics should be explained separately in tasks after the task results.

Github link:
https://github.com/myrahhussain/CA_Labs_MyrahHussain_RomaisaShazad

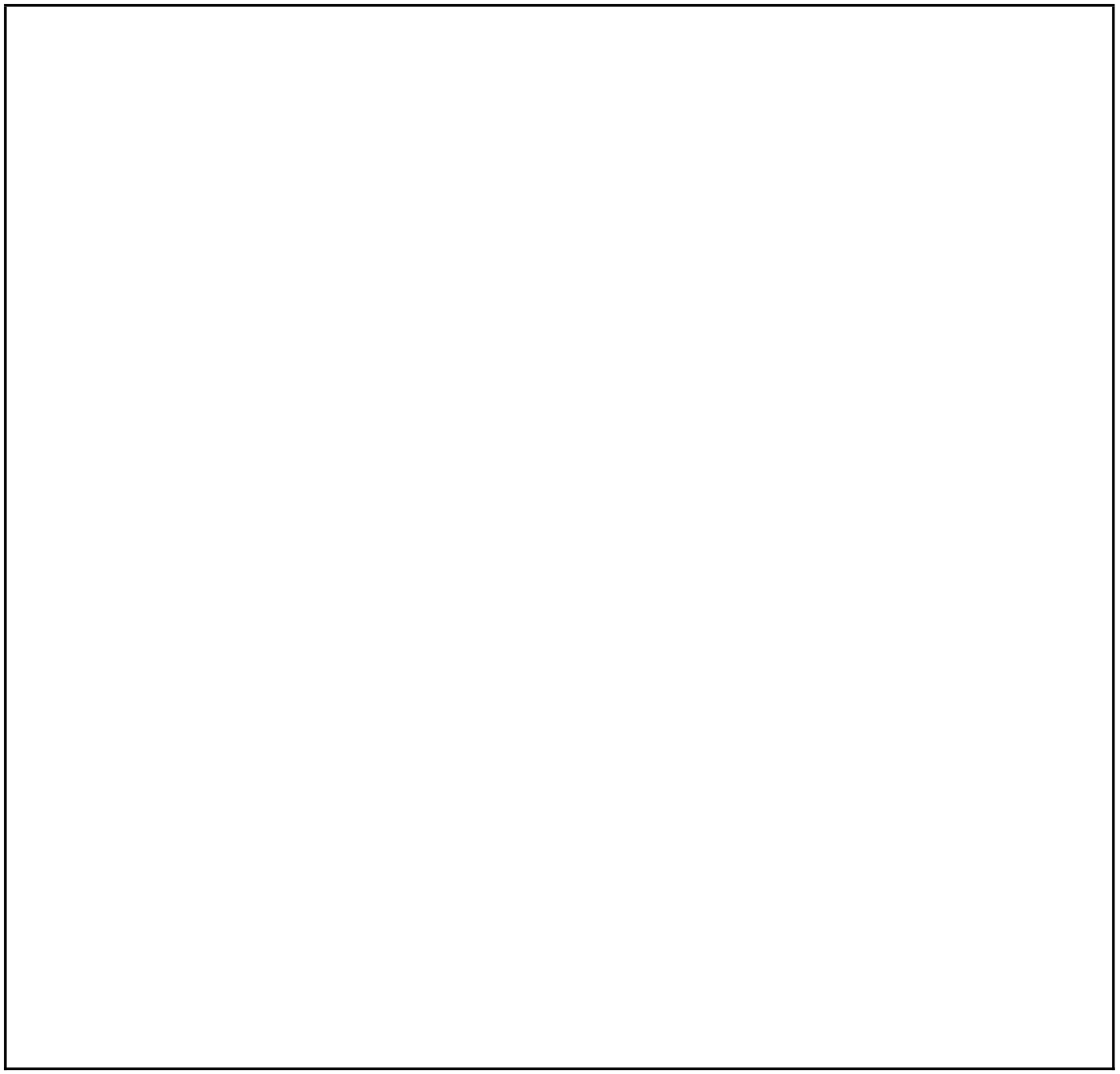
Task 1

FSM of assembly implementation of FSM designed in Lab 05



The FSM has two states: input waiting and countdown.

- In the input waiting state, the LEDs are cleared to zero, and the system keeps reading the switch input in a loop
- If the switches are all zero, it just stays in that loop and keeps checking
- The reset button is also checked here — if it's pressed, the system just stays in idle anyway, since nothing is running
- Once the switch input goes non-zero, the value gets captured and passed to the countdown subroutine as an argument
- In the countdown state, the current counter value is shown on the LEDs at every step
- The counter decrements by one each iteration, with a small delay loop in between so the changes are actually visible
- The reset button is checked on every single iteration of the countdown, not just at the start
- If reset is pressed mid-countdown, execution immediately jumps out of the subroutine, clears the LEDs, restores the stack, and goes back to idle
- When the counter naturally reaches zero, the LEDs are cleared, the stack is restored properly, and the system returns to the input waiting state on its own
- The key difference from Lab 5 is that these states are not hardware flip-flops anymore, they are just labels in assembly code, and the transitions between them are branch instructions



Task 2

Complete assembly source file implementing the FSM.

- The assembly program uses memory-mapped I/O to talk to the hardware — instead of connecting wires directly, specific memory addresses are used to read switches and write to LEDs
- At the start, registers are set up to hold the addresses for switches (768), LEDs (512), and the reset button (1024) so they can be reused throughout the program without recalculating
- The stack pointer is also initialized here to address 511, which is where the stack will grow downward from when functions are called
- The `IDLE_STATE` label marks the beginning of State 1 — it clears the LEDs by writing zero to the LED address, then falls into the read input loop

- `READ_INPUT` is a polling loop that first checks the reset button, then checks the switch value — if both are zero it just keeps looping back to itself
- Once a non-zero switch value is detected, it gets copied into x10 which acts as the argument to the countdown function, then `jal` is used to call `RUN_COUNTER` and save the return address into x1
- `RUN_COUNTER` starts by pushing a stack frame — the stack pointer moves down by 8 bytes to make room, then the return address (x1) and the register x12 are both saved onto the stack before anything else is touched
- This is important because x12 is used as the counter variable inside the function, so its original value needs to be preserved for whoever called it
- The counter starts at the captured switch value and the `DECREMENT_LOOP` shows it on the LEDs, checks if it hit zero, checks the reset button, then decrements and runs a short delay loop before going again
- The delay loop `WAIT_LOOP` runs 3 iterations and also checks reset inside it — this means reset is being checked multiple times per count step, not just once, which makes the response feel immediate
- When the counter reaches zero naturally, `EXIT_COUNTER` clears the LEDs, restores x12 and x1 from the stack, moves the stack pointer back up, and uses `ret` to return to the caller
- If reset is pressed at any point during the countdown, `RESET_ABORT` does the exact same stack cleanup, but instead of returning, it jumps directly to `IDLE_STATE`
- Both exit paths restore the stack in the same way — the same 8 bytes that were allocated at the start are always freed before leaving, so the stack never leaks

```
# Lab 10 FSM Assembly
# Register usage:
#   x2  = stack pointer (SP)
#   x5  = SWITCH_ADDR  = 0x300 = 768
#   x6  = LED_ADDR     = 0x200 = 512
#   x7  = RESET_ADDR   = 0x400 = 1024
#   x10 = captured switch value (argument to RUN_COUNTER)
#   x11 = temp for switch read
#   x12 = current counter value
#   x13 = delay counter
#   x28 = test value
#   x30 = reset register read
#   x1  = return address (ra)

    addi x28, x0,  5      # test value = 5
    addi x2,  x0,  511    # SP = 511
    addi x5,  x0,  768    # SWITCH_ADDR = 0x300
    addi x6,  x0,  512    # LED_ADDR    = 0x200
    addi x7,  x0,  1024   # RESET_ADDR  = 0x400
    sw   x28, 0(x5)       # initial write to switches

IDLE_STATE:
    sw   x0,  0(x6)       # clear LEDs

READ_INPUT:
    lw   x30, 0(x7)       # read reset register
    bne  x30, x0, IDLE_STATE   # if reset pressed, go idle

    lw   x11, 0(x5)       # read switches
    beq  x11, x0, READ_INPUT   # if SW=0, keep waiting

    add  x10, x11, x0    # capture switch value into x10
    jal  x1,  RUN_COUNTER # call countdown
    beq  x0,  x0, IDLE_STATE  # unconditional jump back to idle

RUN_COUNTER:
    addi x2,  x2,  -8    # allocate stack frame
    sw   x1,  4(x2)      # save return address
    sw   x12, 0(x2)      # save x12
```

```
    add  x12, x10, x0     # x12 = counter = captured value

DECREMENT_LOOP:
    sw   x12, 0(x6)       # display counter on LEDs
    beq  x12, x0, EXIT_COUNTER  # if counter=0, exit

    lw   x30, 0(x7)       # check reset
    bne  x30, x0, RESET_ABORT  # if reset, abort

    addi x12, x12, -1     # decrement counter
    addi x13, x0,  3      # delay counter = 3

WAIT_LOOP:
    addi x13, x13, -1     # decrement delay
    lw   x30, 0(x7)       # check reset during delay
    bne  x30, x0, RESET_ABORT
    bne  x13, x0, WAIT_LOOP   # loop until delay done
    beq  x0,  x0, DECREMENT_LOOP  # next count step

RESET_ABORT:
    sw   x0,  0(x6)       # clear LEDs
    lw   x12, 0(x2)       # restore x12
    lw   x1,  4(x2)       # restore return address
    addi x2,  x2,  8      # deallocate stack frame
    j IDLE_STATE          # jump to idle

EXIT_COUNTER:
    sw   x0,  0(x6)       # clear LEDs
    lw   x12, 0(x2)       # restore x12
    lw   x1,  4(x2)       # restore return address
    addi x2,  x2,  8      # deallocate stack frame
    ret
```

Task 3

*Design module code of **instruction memory module** containing memory initialized with assembly instructions:*

Module instruction memory:

- The instruction memory module acts as a read-only program storage — it holds all the assembly instructions and gives the processor one instruction at a time based on whatever address the program counter sends it
- The module takes a 32-bit input called `instAddress` which comes directly from the program counter, and outputs a 32-bit instruction back to the processor every time the address changes
- Inside the module, memory is declared as 256 individual bytes — this is because RISC-V instructions are stored and addressed one byte at a time even though each instruction is 4 bytes wide
- The `$readmemh` command loads the `instruction.mem` file into that byte array at the start of simulation — this is how the machine code gets into the memory without having to hardcode every single byte manually in Verilog
- In the `always` block, four consecutive bytes starting from `instAddress` are combined into one 32-bit instruction — byte 3 goes in the most significant position and byte 0 goes in the least significant, which matches the little-endian format that RISC-V uses
- The `instruction.mem` file contains the machine code translation of the assembly program written in Task 2 — each group of 4 bytes on a line corresponds to one assembly instruction
- For example the first line `13 0E 50 00` is the machine code for `addi x28, x0, 5` and the second line `13 01 F0 1F` is `addi x2, x0, 511` which sets up the stack pointer

- The file is repeated twice because the memory is 256 bytes and the program is around 156 bytes — the second copy just fills the remaining space and does not affect execution since the program counter never reaches that far
- The module is purely combinational — there is no clock, meaning the instruction output updates immediately whenever the program counter changes, which is exactly what a single-cycle processor needs

```
`timescale 1ns / 1ps

module instructionMemory#(
   parameter OPERAND_LENGTH = 31
)(
   input [OPERAND_LENGTH:0] instAddress,
   output reg [31:0] instruction
);

   reg [7:0] memory [0:255];

   initial begin
      $readmemh("instruction.mem", memory);
   end


   always @(*) begin
      instruction = {memory[instAddress + 3],
            memory[instAddress + 2],
            memory[instAddress + 1],
            memory[instAddress]};
   end
Endmodule
```

Instruction.mem memory file:

```
13 0E 50 00
13 01 F0 1F
93 02 00 30
13 03 00 20
93 03 00 40
23 A0 C2 01
23 20 03 00
03 AF 03 00
E3 1C 0F FE
83 A5 02 00
E3 8A 05 FE
33 85 05 00
```

```
EF 00 80 00
E3 02 00 FE
13 01 81 FF
23 22 11 00
23 20 C1 00
33 06 05 00
23 20 C3 00
63 0E 06 02
03 AF 03 00
63 10 0F 02
13 06 F6 FF
93 06 30 00
93 86 F6 FF
03 AF 03 00
63 16 0F 00
E3 9A 06 FE
E3 0C 00 FC
23 20 03 00
03 26 01 00
83 20 41 00
13 01 81 00
6F F0 5F F9
23 20 03 00
03 26 01 00
83 20 41 00
13 01 81 00
67 80 00 00
13 01 F0 1F
93 02 00 30
13 03 00 20
93 03 00 40
23 A0 C2 01
23 20 03 00
03 AF 03 00
E3 1C 0F FE
83 A5 02 00
E3 8A 05 FE
33 85 05 00
EF 00 80 00
E3 02 00 FE
13 01 81 FF
23 22 11 00
23 20 C1 00
33 06 05 00
23 20 C3 00
```

```
63 0E 06 02
03 AF 03 00
63 10 0F 02
13 06 F6 FF
93 06 30 00
93 86 F6 FF
03 AF 03 00
63 16 0F 00
E3 9A 06 FE
E3 0C 00 FC
23 20 03 00
03 26 01 00
83 20 41 00
13 01 81 00
6F F0 5F F9
23 20 03 00
03 26 01 00
83 20 41 00
13 01 81 00
67 80 00 00
```

Lab 10: State-Based Control Flow Using RISC-V Assembly

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/20		
Task 3	/20	-		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.